

AI Agents Practice Projects Portfolio (EN)

Date: 2026-02-23

Repository root path: /workspace

1) Project at a glance

AI Agents Practice Projects is a collection of 18 independent projects designed to build practical capabilities in AI agent architecture and implementation.

Instead of focusing on only one framework, this repository is structured to solve common real-world service problems with multiple stacks and patterns.

- Multi-agent architecture design
- State-based workflow implementation
- Tool integration and automated execution loops
- Human-in-the-loop interaction design
- Guardrail integration (input/output safety)
- Practical runtime interfaces (API, web app, and voice)

Source: README.md (Overview, Learning Objectives, Projects sections)

2) Problem statement and project goals

Problem statement

Many AI agent learning resources are framework-specific and do not make it easy to practice production-relevant concerns (routing, testing, deployment, and safety) in one place.

Goal

This repository is designed to close that gap through:

1. Cross-framework learning: LangGraph / CrewAI / Google ADK / OpenAI Agents SDK / AutoGen
2. Architecture pattern mastery: Supervisor, Handoff, Parallel, Stateful routing
3. Production-style deliverables: Streamlit apps, FastAPI APIs, and voice pipelines
4. Built-in quality management: tests, score-based rewriting loops, and guardrails

3) Repository structure and design philosophy

Structural characteristics

- Each directory is an independently runnable project unit
- Shared practice rules: uv-based dependency management and .env-based secret isolation
- Combines focused concept demos and practical applied examples in one codebase

Root project list (18 total)

1. hello-langgraph
2. tutor-agent
3. multi-agent-architectures
4. youtube-thumbnail-maker
5. workflow-architectures
6. email-refiner-agent
7. workflow-testing
8. content-pipeline-agent
9. job-hunter-agent
10. news-reader-agent
11. a2a
12. financial-analyst
13. youtube-shorts-maker
14. chatgpt-clone
15. customer-support-agent
16. deployment
17. deep-research-clone
18. my-first-agent

Source: README.md (Projects, Project Structure)

4) Technology stack details

Shared foundation

- Language: Python
- Package manager: uv
- Environment handling: .env (API key separation)

Framework-specific strengths

A. LangGraph projects

- StateGraph-centered, stateful flow design

- Conditional routing, parallel send, checkpoint persistence
- Representative projects: hello-langgraph, tutor-agent, youtube-thumbnail-maker, workflow-testing

B. CrewAI projects

- Role-based agents + task chains
- YAML-based separation of agents/tasks for maintainability
- Score-based quality gating with rewrite loops
- Representative projects: content-pipeline-agent, job-hunter-agent, news-reader-agent

C. Google ADK projects

- Sub-agent tool composition
- Parallel execution with ParallelAgent
- Structured outputs and artifact generation
- Representative projects: youtube-shorts-maker, financial-analyst, a2a

D. OpenAI Agents SDK projects

- Handoff, guardrails, and session management
- Streamlit UI with streaming output handling
- Voice pipeline support
- Representative projects: chatgpt-clone, customer-support-agent, deployment

E. AutoGen project

- Team-based research collaboration pattern
- Multi-agent research loops with planning, investigation, and termination conditions
- Representative project: deep-research-clone

5) Core implementation patterns (detailed)

5-1. Human-in-the-loop improvement loops

- User feedback is treated as a workflow control signal, not just an input string
- Example: poem generation feedback followed by iterative refinement
- Value: practical quality tuning for generative outputs

Related projects:

- hello-langgraph
- youtube-thumbnail-maker

5-2. Router-based multi-agent systems

- A classification node evaluates user context and delegates to specialized agents

- Example: learner-level classification -> teacher/feynman/quiz branch
- Value: better response quality plus clear responsibility boundaries in complex domains

Related projects:

- tutor-agent
- multi-agent-architectures
- customer-support-agent

5-3. Parallel generation + aggregation

- Generates multiple candidate outputs in parallel, then aggregates/selects
- Example: multi-candidate thumbnail generation and chunk-level parallel summarization
- Value: lower latency and higher output diversity

Related projects:

- youtube-thumbnail-maker
- workflow-architectures
- youtube-shorts-maker

5-4. Quality-gated automatic retries

- Automatically rewrites output if score thresholds are not met
- Enforces quality through operational rules
- Value: moves from one-shot generation to iterative quality-improvement pipelines

Related projects:

- content-pipeline-agent

5-5. Built-in guardrails and safety

- Applies input/output filtering to block risky responses
- Value: essential safety baseline for production use

Related projects:

- customer-support-agent

6) Top 10 representative project summary

Project | Problem being solved | Core flow | Technical highlights

--- | --- | --- | ---

hello-langgraph | LangGraph fundamentals | Conversation + feedback loop | interrupt, checkpoint

tutor-agent | Personalized learning support | Classification -> expert-agent routing | conditional edges

multi-agent-architectures | Multilingual customer support | Supervisor delegates to language-specific agents |

AgentTool pattern

youtube-thumbnail-maker | Video-driven thumbnail automation | Extract/summarize -> parallel candidates -> feedback selection | Send pattern, external tool integration
workflow-testing | Agent quality validation | E2E + node-level + evaluation-based tests | standardized testing strategy
content-pipeline-agent | Automated marketing content production | Research -> generation -> scoring -> rewriting | CrewAI Flow, scoring gate
job-hunter-agent | Job search process automation | matching/selection/resume/company research/interview prep | YAML config, multi-stage chain
customer-support-agent | Voice customer support | triage -> specialized agent handoff | VoicePipeline, guardrails
chatgpt-clone | General AI assistant UI | multi-tool execution + streaming responses | Streamlit, MCP integration
deployment | API-ready deployment model | conversation/message/streaming APIs | FastAPI, SSE, deployment config

7) Data flow from an architecture perspective

A. Stateful routing pattern (example: tutor-agent)

1. User message enters the system
2. Classification agent determines learner level
3. Request is routed to the proper specialist agent
4. Results are accumulated in state and returned as final output

Key point:

- Classification output acts as a routing key for next-node selection, not just a label

B. Quality evaluation loop pattern (example: content-pipeline-agent)

1. Research data is collected
2. Draft content is generated
3. SEO/virality scores are evaluated
4. If below threshold, rewriting is triggered
5. If passed, final output is confirmed

Key point:

- Retry conditions are encoded directly as pipeline rules

C. Voice interface pattern (example: customer-support-agent)

1. Voice input is captured
2. Speech is converted and intent is analyzed
3. Triage agent identifies the specialist domain
4. Assigned agent generates the resolution response
5. Output guardrails validate response before voice output

Key point:

- Balances user experience (voice), safety (guardrails), and extensibility (handoff)

8) Maturity across execution, testing, and deployment

Execution

- Run each project independently after uv sync
- Streamlit apps: `uv run streamlit run main.py`
- Python apps: `uv run python main.py`

Testing

- workflow-testing includes E2E tests, node-level tests, and evaluation-based patterns
- Explicitly treats workflow behavior itself as a testable artifact

Deployment

- deployment provides FastAPI + SSE support
- `railway.json` offers a direct cloud deployment starting point

9) Portfolio strengths (highlight points)

1. Cross-framework capability
 - Demonstrates framework selection and implementation by context, not tool lock-in
2. Architecture-pattern depth
 - Implements Supervisor, Handoff, Parallel, Routing, and Human-in-the-loop in working code
3. Production-facing interface experience
 - Covers Streamlit web apps, VoicePipeline, and FastAPI APIs
4. Built-in quality and safety
 - Includes score-based rewrites, guardrails, and testing strategy patterns
5. Scalable structure
 - Independent project modules + YAML-driven configuration support maintainability

10) Improvement roadmap (future expansion)

1. Shared observability layer
 - Standard log schema, trace IDs, and latency metrics across projects
2. Unified benchmark dashboard

- Automated cross-project comparison for quality/cost/latency
3. Stronger CI pipelines
 - PR-level regression testing and quality gates for agent workflows
 4. Standardized tool interfaces
 - Shared tool specs to improve portability and reusability between projects
 5. Expanded deployment templates
 - Add Docker/Kubernetes examples beyond Railway

11) One-line summary for interviews/presentations

"This repository is a practical AI agent portfolio that compares and implements core patterns (routing, parallelism, state management, safety, and deployment) across frameworks through 18 independent projects."

12) Primary references used

- README.md
- hello-langgraph/README.md
- tutor-agent/README.md
- multi-agent-architectures/README.md
- youtube-thumbnail-maker/README.md
- content-pipeline-agent/README.md
- job-hunter-agent/README.md
- customer-support-agent/README.md
- workflow-testing/README.md
- deployment/README.md
- a2a/README.md
- each project's pyproject.toml